DB25 Execution Engine Simulator — Design

> Status: design accepted, implementation not started. Increments S0–S6 below. Companion to `physical-planner.md`, which defines the seam this component fills.

---

1. Why (the one page)

The physical planner is a pure function of three declared inputs (`physical-planner.md` §1). Two of them are currently placeholders:

Input	What it should carry	What it carries today
`CalibrationProfile`	hardware-derived cost coefficients	hand-reasoned ratios; `simd_width`, `cache_line`, `cluster_nodes` are *informational only*
`ExecutionCapabilityProfile`	what the engine can do and what the hardware **is**	a list of operator names
`RuntimeProfile`	observed cardinalities, timings, memory, spills	`struct RuntimeProfile {};` — the type and the seam, no producer

The seam was designed correctly and left unfilled, on the stated grounds that the producer "arrives with the execution engine." That was the right call at the time and it is now the binding constraint: **every remaining roadmap item is blocked on a component we are not going to build next.** The fast path needs a planning-time budget nobody can derive. Adaptive re-planning needs a profile nobody produces. `exchange_row = 12.0` is a number with a good argument behind it and no measurement.

So we build the producer explicitly, and we call it what it is: an **execution engine simulator**. It does not execute queries. It answers one question, from declared hardware physics:

> Given this physical plan, on this machine, over this data — where does the time go, where does the memory go, and what would a real engine have observed?

That answer fills all three inputs. It also does something a real engine could not do as cheaply: it answers the question **for a machine we do not own**, which is the difference between a planner tuned for one lab box and a planner that can be asked "what would this plan cost on CXL-attached memory?"

The hazard, stated first

A simulator built out of the cost model's own coefficients is a machine for agreeing with the cost model. `Σ coefficient × rows` simulated by `Σ coefficient × rows` proves nothing, and "the optimizer picked the fastest plan" becomes a tautology dressed as a result.

So the load-bearing decision of this whole component is that the simulator and the cost model do not share a single parameter. The simulator is parameterized by *device physics* — bandwidth, latency, capacity, concurrency, clock, issue width. The cost model's coefficients are **derived from** the simulator by a stated procedure (S5). The two are then related by a derivation that can be checked, and where they disagree, the disagreement is a **finding** rather than a bug in one of them.

That inverts the usual failure mode. We are not building a simulator to confirm the cost model. We are building it to **find the queries where a linear per-row model cannot see what the hardware does** — and every one of those is a defect report with a reproduction.

---

2. Design decisions

Each decision is a choice + the rationale + what it rules out.

E1 — Its own repository, and the dependency arrow only points one way

`db25-execution-sim` depends on `db25-physical-plan`. Nothing depends on it except tooling and the umbrella.

- **Why:** the planner's purity is the whole game. If the simulator lived inside the planner repo, "the planner must not call into a running system" would be a matter of discipline; as a separate repo with the dependency pointing inward it is structural. The simulator *consumes* plans and *produces* the planner's typed inputs; it is never on the planning path.
- **Rules out:** the convenient shortcut where a cost function peeks at a simulated timing. It cannot — the header is not there.

E2 — YAML for the machine model, deliberately not the s-expr IDL

The operator spec stays `physical.spec.sexpr`. The machine model is `machines/*.yml`.

- **Why:** they have different authors and different lifetimes. The spec is an IDL the planner *interprets*, versioned with the code, generating conformance tests. A machine model is a **datasheet transcription** — the person editing it is reading a Xeon spec sheet or an NVMe product brief, and YAML is the register that audience already writes in.
- **Rules out:** one format for everything. That is a real cost and it is accepted.

E3 — A strict YAML subset, hand-written, with explicit units

No `yaml-cpp`, no anchors, no flow style, no multi-document, no implicit typing. Every physical quantity is a **string with a unit**: `bandwidth: 7 GB/s`, not `bandwidth: 7000000000`.

- **Why (dependency):** the repo family is zero-dependency and `-fno-exceptions`. A strict subset reader is a few hundred lines and it is *ours* to make strict.
- **Why (units):** this is the correctness argument, and it is the stronger one. YAML's implicit typing is a documented source of silent misreads — `NO` becomes false, `1:30` becomes 90, `0x10` becomes 16, a trailing `f` makes a string. A machine model is nothing *but* numbers, and a machine model that silently misreads one produces a

plausible wrong answer. Parsing `7 GB/s` ourselves routes every physical quantity around YAML's type coercion entirely.

- **Also:** it forces the unit to be *written down*, which settles the decimal-versus-binary question at the point of entry. `GB` is 10^9, `GiB` is 2^{30}, and a model that confuses them is 7% wrong before it starts. Storage vendors quote decimal; memory is binary; the file says which.
- **Rules out:** pasting arbitrary YAML from elsewhere. Unknown keys are an **error**, not a shrug — a typo'd parameter that silently keeps its default is exactly how a simulator lies to you.

E4 — Every number carries its provenance, and the simulator reports how much it made up

Each parameter is `{ value: <quantity>, source: datasheet|measured|derived|assumed, note: "" }`.

- **Why:** this is the scientific claim of the whole component. A simulated timing whose critical path rests 80% on `assumed` numbers is a different epistemic object from one resting 95% on `datasheet`, and a tool that reported both as "412 µs" would be misleading its user by omission.
- **What it buys:** every result carries a **confidence decomposition** — for this answer, what fraction of the critical path came from measurements, and which assumed parameters is it most sensitive to. That last part is a one-at-a-time sensitivity sweep, and it turns "I don't trust this number" into "this number moves 30% when `overlap_efficiency` moves 10%, and `overlap_efficiency` is a guess."
- **Rules out:** unattributed numbers. A parameter with no `source` fails to load.

E5 — Little's law capped by peak, as the single bandwidth primitive

For any tier or link:

```
bw_concurrency = concurrency × granularity / latency        (Little's law)
bw_achieved    = min(peak_bandwidth, bw_concurrency)
```

- **Why:** this one formula is what actually distinguishes the devices, and it distinguishes them *correctly*:

device	latency	granularity	peak	in-flight needed for peak
DRAM	~85 ns	64 B	~460 GB/s	~610 lines
CXL 3.0 type-3	~320 ns	64 B	~64 GB/s	~320 lines
NVMe Gen5 (×4)	~12 µs	4 KiB	~28 GB/s	~82 requests
NVLink 5	~1.8 µs	large	~900 GB/s	bandwidth-bound
100 GbE	~25 µs	~9 KiB	~12.5 GB/s	~35 requests

Read that table as the answer to *why CXL changes plans*: it is not slow, it is **latency-expensive relative to its bandwidth**, so a kernel that keeps many requests in flight (a streaming scan) gets nearly its full bandwidth, and a kernel that chases pointers one at a time (a hash probe with no batching) falls off a cliff. Two operators, same device, an order of magnitude apart. A per-row coefficient cannot say that. This formula says it in one line.

- **Rules out:** modelling a device by its bandwidth alone, which is the mistake that makes NVMe look like slow DRAM instead of like something categorically different.

E6 — Residency as a hit rate, not a cliff

A kernel's working set `W` against a cache of capacity `C` under random access: `hit_rate ≈ min(1, C/W)`, and effective latency is the blend.

- **Why not a hard cliff:** a hash table 10% larger than L3 does not become DRAM speed; 90% of its probes still hit. A step function would be *more* confident and *less* correct.
- **Why this matters at all:** it is the primary source of **non-linearity in build-side size**, and therefore the primary reason a linear cost model gets a hash join wrong. Doubling the build side does not double the probe cost — it can quadruple it as the table falls out of L3. Finding those is S6's job.
- **Stated as an approximation.** It assumes uniform random access with LRU. It is wrong for a sequential scan (which the prefetcher handles and which we model separately) and wrong for a skewed probe distribution (which *helps*, and which we take credit for only when the data model says so).

E7 — The simulator is not a correctness oracle

It computes time, bytes and memory. It never computes a result set. Query results are checked by the frontend's existing golden corpus, not here.

- **Why say it:** a component called "execution engine simulator" invites the assumption that it validates answers. It does not, and a reader who believed it did would stop looking for a real validation path.

E8 — Relative ranking is the product; absolute time is a by-product

The simulator's contract is that it **orders plans correctly**. Its absolute microsecond figures are reported, and are reported with their provenance decomposition attached, precisely so they are harder to quote out of context.

- **Why:** absolute accuracy against a real engine is unachievable without the engine, and claiming it would be the dishonest version of this project. Ranking is achievable, is what a planner actually needs, and is falsifiable today against closed-form cases and against the real hardware we do have.

---

3. The physics

Four models, composed. Each is small enough to test in isolation, which is the point of separating them.

3.1 The resource model

A machine is a set of **resources**: compute units, memory tiers, links. Each has `capacity`, `latency`, `peak_bandwidth`, `granularity`, `concurrency`, and a `scope` (per-core / per-socket / host / cluster). E5 turns a request stream into an achieved bandwidth. Scope is what makes contention real: two cores sharing an L3 contend; two cores with private L1s do not.

3.2 The kernel model

For each of the 29 physical operators, a **kernel**: what it reads, what it writes, what state it holds, and how big that state is. Not a coefficient — a function of the machine and the data sizes.

```
t_compute = ops / (cores × ipc × simd_lanes × clock)
t_memory  = Σ_tier bytes_tier / bw_achieved(tier)
t_kernel  = max(t_compute, t_memory) + (1 − η) × min(t_compute, t_memory)
```

`η` is `overlap_efficiency` — how much of the memory time hides under compute. It is an **assumed** parameter and is the one the sensitivity sweep will flag most often. That is correct: it is the biggest guess in the model and the report should say so every time.

Kernels are declared in the spec alongside `edges`, so **adding an operator forces someone to say what it does to the machine** — the same discipline that made nine wrong pipeline labels visible in Increment 4.1.

3.3 The schedule model

`pipelines()` from Increment 4.1 already returns pipelines in dependency order. The scheduler walks them:

- A `Materialized` **edge is a barrier** and a memory high-water contribution.
- A `Rescanned` **edge multiplies** its subtree's cost by the outer cardinality — the same `input_evaluations` logic the cost model learned for recursion.
- **Parallelism:** `P` workers per pipeline, where `P` is `CapabilityProfile::max_dop` — a declared capability of the engine. It is `1` today. See "the serial fraction" below, which is where the real content is.
- **Contention:** concurrently running pipelines share each resource's bandwidth in proportion to demand, capped at that resource's peak.
- **Spills:** when a `Materialized` edge's state exceeds the largest non-durable tier, it spills, and the spill is *charged at the NVMe tier's parameters*. This is where the memory hierarchy stops being an efficiency question and starts being a plan-choice question.

The serial fraction, and why `t_total / P` is not a model

`P` is `CapabilityProfile::max_dop`: a **declared capability of the engine**, not a number the simulator invents and not one the plan carries. It is `1` today, which is what DB25 actually is. A simulator that assumed more would be timing an execution the planner never costed, and the estimated-versus-observed comparison this whole document exists for would be comparing two different machines.

A pipeline finishes when its slowest worker does, so:

```
t = W × (s + (1 − s) / P) × skew
```

`W` is the pipeline's total work and `s` **is its serial fraction**, derived from the declared `parallelism` class of the operators in it — `full` contributes 0, `partitioned` contributes its partitioning step, `ordered` its within-group work, `serial` contributes 1. `skew` comes from the data model, not from a constant.

The serial fraction is the whole point, and `t = t_total / P` is the version of this formula that cannot work. Dividing by `P` makes every pipeline equally parallel, so it rescales all plans by the same factor and can never change which plan wins — and changing which plan wins is the only effect parallelism has on a planner. With `s` present, a costlier plan overtakes a cheaper one exactly when $s_2/s_1 < W_1/W_2$: the break-even is the cost ratio, so the simulator can state what a plan's parallelism would have to be worth before it changes the answer. That is a falsifiable claim about the cost model. A flat `W/P` is not a claim at all.

The `parallelism` field this reads was declared on all 29 operators in Increment 4 for exactly this purpose — `spec.hpp` says so in as many words — and nothing in the cost model reads it. The simulator is its first consumer, which makes the simulator its first test.

3.4 The data model

A separate `datasets/*.yml`: per-column NDV, distribution (uniform / zipf(θ) / normal), null fraction, correlations, row width, value widths.

- **This is not optional garnish.** Without it the simulator's "observed" cardinalities would be the cost model's *estimated* cardinalities, the `RuntimeProfile` would teach the planner nothing, and adaptive re-planning would have nothing to adapt to.
- **With it,** the divergence between what the planner *estimated* and what the simulator *observed* is exactly the signal adaptive execution exists to consume — and it is generated honestly, because the two came from different models.
- It is also where **skew** comes from. A zipf(1.1) join key is why one worker gets 40% of the build side, and that is a plan-relevant fact no cardinality estimate carries.

---

4. Implementation plan

Each increment is independently mergeable, CI-gated, and falsifiability-tested (every fix removed by mutation, confirming the matching test fails) on the standing discipline of this project.

S0 — the machine model and its reader. Strict YAML subset (nested maps, lists, scalars, comments; no anchors, no flow style, no implicit typing). Unit-aware quantity parsing with decimal/binary distinction. Provenance required on every parameter. Unknown key = error, missing required = error. Ships four reference machines — `nvme-server.yml`, `cxl-tiered.yml`, `nvlink-gpu.yml`, `laptop.yml` — plus `minimal.yml`, a deliberately trivial machine whose numbers make hand-computation possible so the tests can check closed-form answers. *Gate:* a round-trip and a distinguishability sweep — change one parameter of one device, require the loaded model to differ; and a coverage check that reads the `MachineModel` struct so a field added without a sweep case fails.

S1 — the resource model and the bandwidth primitive. E5 and E6, in isolation, with no plan in sight. *Gate:* closed-form cases. A device with `concurrency × granularity / latency` above its peak must return its peak; below it must return the Little's law value; the DRAM/CXL/NVMe table in §2 E5 is a test, not a comment.

S2 — operator kernels. A kernel per physical operator, declared in the spec next to `edges`, with the conformance check extended so all 29 are covered. *Gate:* the applicability sweep pattern — an operator with no kernel fails the build's tests, by name.

S3 — the pipeline scheduler. Barriers, rescan multiplication, parallelism, skew, contention, spills. Produces a timeline. *Gate:* a spill must be *visible* — a plan whose build side is grown past the last non-durable tier must show a discontinuity in total time, and the mutation that removes spill handling must be killed by that test. *Second gate, on the parallelism model:* at `max_dop = 1` the scheduler must produce exactly the serial timeline, so the parallel path cannot drift while the engine is serial and nobody notices. And a `MachineModel` asking for a `P` above the capability profile's `max_dop` must be an **error**, not a silently honoured request — the two profiles would otherwise describe different machines and the disagreement report in S6 would be measuring that difference rather than the cost model.

S4 — the `RuntimeProfile` producer. Populate the type the planner already accepts: per-operator observed rows, wall time, bytes by tier, peak memory, spilled bytes; per-pipeline workers and observed skew; totals and critical path; and the provenance decomposition (E4). **This is the increment that unblocks adaptive re-planning**, which has

been the stated blocker since Increment 0. *Gate:* observed cardinalities must diverge from estimated ones on at least one corpus query — if they never do, the data model is not doing its job and the whole feedback loop is decorative.

S5 — calibration derivation. Derive a `CalibrationProfile` from a `MachineModel` by simulating single-operator microbenchmark plans and fitting each per-row coefficient. Makes `simd_width`, `cache_line` and `cluster_nodes` load-bearing instead of informational, and makes `exchange_row` a **derived** number for the first time — it becomes a function of which link the exchange crosses, so NVLink and 100 GbE stop being the same number. *Gate:* the derived profile must reproduce the ratios the hand-reasoned profile argues for, **or** the discrepancy is written down as a finding with the machine that exhibits it. Both outcomes are results; silently adopting either number is not.

S6 — the disagreement report, as a gate. The scientific payoff, and the reason the rest exists. For each corpus query × each machine model: take the candidate plans the memo already enumerated, rank them by the cost model, rank them by the simulator, and report **Spearman rank correlation** plus the **worst inversion** — the pair where the cost model was most confidently wrong. *Gate:* correlation must not fall against a pinned baseline. A change that makes the cost model less physical fails CI. And the inversions are a **work queue**: each one is a cost-model defect that arrives with a machine and a query that exhibit it.

---

5. What this buys, concretely

- `exchange_row` **stops being an assertion.** It becomes a function of the link, so a plan on NVLink and the same plan on Ethernet no longer cost the same.
- **The hash-join build-side choice gets a reason.** Today it is a cardinality comparison. With residency it becomes "the smaller side, *unless the larger one still fits in L3*," which is a different answer and sometimes the right one.
- **Open question 5 becomes answerable.** The fast-path threshold has to be derived from a planning-time budget; a budget is a ratio of planning time to *execution* time, and execution time is precisely what nothing could estimate. S4 estimates it.
- **Adaptive re-planning gets something to adapt to** — an observed cardinality produced by a model the planner does not share.
- **The cost model gets an adversary.** S6 is the first mechanism in this project that can tell us the cost model is wrong *without a human noticing first*.

---

6. Open questions

1. **Validation against real hardware.** The simulator's ranking claim is falsifiable against a real box for the operators we can microbenchmark today (scan, sort, hash build/probe). Do we build that harness in S1, or accept closed-form tests until an engine exists? *Leaning: a* `measure/` *target in S1, because a* `source: measured` *provenance we never actually measure is the exact dishonesty E4 exists to prevent.*
2. **GPU kernels.** `nvlink-gpu.yml` describes the interconnect; it does not describe a GPU execution model (occupancy, warp divergence, shared memory). Does S2 model GPU kernels, or does the GPU machine exist only

to make the *link* cost real? *Leaning: links only, until there is a reason otherwise — modelling GPU occupancy badly is worse than not modelling it.*

3. **When `max_dop` moves above one.** The serial fraction `s` is derived from the operators' declared `parallelism` classes, which is the most honest source available and still a derivation rather than a measurement. The first engine with real workers will make `s` measurable, and the reconciliation between declared and measured `s` is a finding to write down — not a coefficient to quietly retune. Until then `P` is 1 and `s` is inert, which is the correct state for a planner whose cost model is serial by decision.
4. **Where the data model's numbers come from.** Hand-written per corpus table today. Derived from the catalog's histograms when those exist — at which point the estimated/observed divergence narrows, which is the *intended* outcome and must not be mistaken for the loop breaking.